



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



Apache Log4j2 2026 Structured Logging, TLS Appender, and Deserialization Exposure

Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-08-26
Version	1.0
Author	Lost Boys Cyber
Report Type	Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | Apache Log4j2 2026 Structured Logging, TLS Appender, and Deserialization Exposure - Vulnerability Threat Intelligence Report

Published: 2026-08-26 | TLP:CLEAR | Version: 1.0 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Vulnerability Metadata
3. Source Review & Confidence
4. Vulnerability Details
 - 4.1. Structured JSON issues: CVE-2026-49844 and CVE-2026-34481
 - 4.2. Structured XML issues: CVE-2026-34480 and CVE-2026-34479
 - 4.3. Syslog/RFC5424 log injection: CVE-2026-34478
 - 4.4. TLS appender hostname verification: CVE-2026-34477 and CVE-2025-68161
 - 4.5. Emerging deserialization receiver issue: GitHub issue #4255
5. Attack Scenarios
6. Threat Landscape & Exploitation Status
7. Affected Products and Exposure
8. Mitigation and Workarounds
9. Detection Engineering
 - 9.1. Detection Summary
10. Threat Hunting
 - 10.1. Hunt Hypotheses
11. MITRE ATT&CK Mapping
12. Validation / Lab Testing
13. Recommended Actions Summary
14. Indicators, Observables, and Source Limitations
15. References

Apache Log4j2 2026 Structured Logging, TLS Appender, and Deserialization Exposure

1. Executive Summary

BLUF: As of the 2026-08-26 source review, Lost Boys Cyber did not identify an Apache-published CVE advisory newly released on 2026-08-26, but did identify a same-day public disclosure and GitHub issue for Apache logging-log4j2 issue #4255: a reported FilteredObjectInputStream allowlist bypass via java.rmi.MarshalledObject that may enable Java deserialization RCE in applications receiving serialized LogEvent objects over the network. The report now treats #4255 as an emerging, unassigned, not-yet-vendor-advisory vulnerability candidate alongside the Apache-published 2026 structured logging and TLS appender cluster.

This is not Log4Shell redux. The reviewed 2026 Log4j2 issues are mostly medium-severity conditions that require specific layouts, appenders, log message types, or native Java serialized log-event receiver services. They can still matter operationally because modern detection, SIEM, compliance, and incident-response workflows assume logs are well-formed and reliably delivered. A malicious input that breaks JSON/XML output, injects syslog record boundaries, suppresses a record, or permits interception of remote appender traffic can reduce audit reliability exactly when defenders need telemetry most.

Immediate defender action: inventory Log4j artifacts across application images and build pipelines, upgrade Log4j API/Core/Layout Template JSON components to fixed releases, prioritize systems using JsonTemplateLayout, XmlLayout, Rfc5424Layout, Socket/Syslog/SMTP appenders with TLS, and the deprecated Log4j 1-to-2 bridge. Where upgrades are delayed, reduce exposure by changing layouts away from affected structured output paths, hardening TLS trust stores, and monitoring for parsing failure spikes or missing telemetry around suspicious requests.

2. Vulnerability Metadata

CVE / Issue	Component	Affected Versions	Fixed Version / Recommended Target	Severity	Primary Impact
CVE-2026-49844	org.apache.logging.log4j:log4j-api	2.13.1 through 2.25.4, 2.26.0, and 3.0.0-alpha1 through 3.0.0-beta2	2.25.5 or 2.26.1	CVSS v4 6.3 Medium	Invalid JSON from MapMessage.asJson() with NaN/Infinity values can corrupt or disrupt log ingestion
CVE-2026-34481	log4j-layout-template-json	2.14.0 through 2.25.3 and 3.0.0-alpha1 through 3.0.0-beta3	2.25.4, but prefer 2.25.5+ because CVE-2026-49844 covers remaining code paths	CVSS v4 6.3 Medium	Invalid JSON from JsonTemplateLayout with non-finite floating-point values
CVE-2026-34480	log4j-core XmlLayout	2.0-alpha1 through 2.25.3 and 3.0.0-alpha1 through 3.0.0-beta3	2.25.4+	CVSS v4 6.9 Medium	Unescaped XML 1.0 forbidden characters can produce malformed XML or silently lose events
CVE-2026-34479	log4j-1.2-api bridge Log4j1XmlLayout	2.7 through 2.25.3 and 3.0.0-alpha1 through 3.0.0-beta2	2.25.4+; eliminate bridge where possible	CVSS v4 6.9 Medium	Malformed XML and downstream record loss in compatibility bridge usage

CVE-2026-34478	log4j-core Rfc5424Layout	2.21.0 through 2.25.3 and 3.0.0- beta1 through 3.0.0- beta3	2.25.4+	CVSS v4 6.9 Medium	CRLF log injection or unexpected framing downgrade due to silent configuration incompatibility
CVE-2026-34477	log4j-core network appenders	2.12.0 through 2.25.3 and 3.0.0- alpha1 through 3.0.0-beta3	2.25.4+	CVSS v4 6.3 Medium	verifyHostName attribute ignored for SMTP, Socket, or Syslog appenders using nested Ssl configuration
CVE-2025-68161	log4j-core Socket Appender	2.0-beta9 through 2.25.2	2.25.3+; prefer 2.25.5+/2.26.1 where compatible	CVSS v4 6.3 Medium	Missing TLS hostname verification in Socket Appender; precursor to CVE-2026-34477 incomplete-fix issue
GitHub issue #4255 / WVE-2026-007- style secondary tracking	log4j-api FilteredObjectInput Stream and log4j- core LogEventProxy serialized event handling	Public secondary reporting claims log4j-api 2.11.0- 2.26.1 and log4j- core 2.8.0-2.26.1; Apache issue was closed and body removed at review time; no CVE observed	No official Apache fixed release or CVE observed at review time; mitigate by eliminating serialized LogEvent receivers and applying JVM serial filters where safe	Unscored / emerging; potentially Critical in exposed receiver architectures	Java deserialization filter bypass through java.rmi.Marshalled Object when a service receives serialized LogEvent objects over a network

3. Source Review & Confidence

Source	Contribution	Confidence
Apache Logging Services security page	Authoritative vendor vulnerability index and threat-model framing	High
Apache CycloneDX Vulnerability Disclosure Report	Machine-readable affected ranges, CVSS vectors, descriptions, recommendations, and publication/update timestamps	High
CVE Program records	CNA details, timelines, affected package coordinates, and reference links	High
Apache mailing-list advisory threads	Advisory publication evidence and direct vendor references	High
Apache Log4j manual pages	Configuration-specific exposure details for JsonTemplateLayout, XmlLayout, Rfc5424Layout, and network appenders	Medium-High
GitHub pull requests #4074, #4075, #4077, #4078, #4080, and #4163	Patch provenance and affected code paths	Medium-High
Public vulnerability-database mirrors and	Corroboration that commercial tools are	Medium

scanner/vendor notices	flagging fixed versions	
The CyberSec Guru article on Log4j2 deserialization RCE, published 2026-08-26	Secondary same-day reporting of issue #4255, affected-version claims, exploit preconditions, and mitigations	Medium; useful lead, but not equivalent to Apache advisory
GitHub apache/logging-log4j2 issue #4255 and public comments, reviewed through GitHub API	Confirms same-day issue existence, closure, deletion of public body, and community links to PoC repositories	Medium; issue metadata is authoritative for repository state, but original body was unavailable
Public PoC repository references for log4j-4255	Evidence that proof-of-concept material is circulating	Medium-Low; code not relied on for exploitation instructions in this report

Analyst confidence is high for affected versions, fixed versions, and exploit preconditions because the core claims are drawn from Apache and CVE CNA material. Confidence is medium for exploitation-in-the-wild because the reviewed public sources did not provide credible evidence of active exploitation against real victims. For issue #4255, confidence is medium that public PoC material is circulating, but lower that Apache has accepted the report as a CVE-class issue because the GitHub issue was closed and its body was unavailable during review.

4. Vulnerability Details

The 2026 Log4j2 cluster is best understood as telemetry-trust degradation rather than general remote code execution. The common theme is that user-influenced content, numeric values, or configuration compatibility issues can cause structured log output to become invalid, downstream parsers to reject records, CRLF-based record injection, or weaker TLS verification for remote logging appenders.

4.1. Structured JSON issues: CVE-2026-49844 and CVE-2026-34481

CVE-2026-34481 affects JsonTemplateLayout when logged objects or MapMessage content include non-finite floating-point values such as NaN, Infinity, or -Infinity. RFC 8259 JSON does not permit these bare tokens, so conformant parsers can reject the resulting log document. CVE-2026-49844 is the follow-on incomplete-fix issue: MapMessage.toJson() in log4j-api still emitted invalid JSON in affected 2.25.4/2.26.0 code paths.

The attacker model is narrow but realistic: an application logs attacker-controlled numeric values through JsonTemplateLayout or another path relying on MapMessage.toJson(). Examples include API request bodies, telemetry values, scoring fields, scientific/ML inputs, or deserialized application objects whose fields can contain non-finite floating-point values.

4.2. Structured XML issues: CVE-2026-34480 and CVE-2026-34479

CVE-2026-34480 affects Log4j Core XmlLayout when log messages or MDC values contain XML 1.0 forbidden characters. With the JRE built-in StAX implementation, forbidden characters may be emitted, producing malformed XML; with alternative StAX implementations such as Woodstox, exceptions can cause a log event not to reach the intended appender. CVE-2026-34479 affects Log4j1XmlLayout in the Log4j 1-to-2 bridge and can produce malformed XML for bridge users or applications relying on Log4j 1 compatibility configuration.

4.3. Syslog/RFC5424 log injection: CVE-2026-34478

CVE-2026-34478 affects direct Rfc5424Layout users in Log4j Core 2.21.0 through 2.25.3. Undocumented renames of security-relevant configuration attributes could silently disable newline escaping for TCP framing or silently downgrade TLS framing to unframed TCP without newline escaping. A user able to inject CRLF sequences into logged content may be able to alter syslog record boundaries or create confusing downstream records. Apache states that SyslogAppender users are not affected by this specific issue because its configuration attributes were not modified.

4.4. TLS appender hostname verification: CVE-2026-34477 and CVE-2025-68161

CVE-2025-68161 identified missing TLS hostname verification in Socket Appender. CVE-2026-34477 covers an incomplete-fix pathway where the verifyHostName attribute in a nested Ssl element was silently ignored for SMTP, Socket, or Syslog

appenders. A network-positioned attacker may be able to intercept or redirect remote log traffic if they can present a certificate issued by a CA trusted by the appender trust store or Java default trust store.

4.5. Emerging deserialization receiver issue: GitHub issue #4255

Same-day secondary reporting describes a Log4j2 Java deserialization issue tracked in Apache logging-log4j2 as issue #4255. The claimed flaw is a FilteredObjectInputStream allowlist bypass where java.rmi.MarshalledObject is allowed as an outer serialized class, but its inner objBytes payload is later deserialized through a plain ObjectInputStream when Log4j reconstructs a serialized LogEvent message. Secondary reporting claims this can lead to RCE when an exposed service receives serialized LogEvent objects over a network and the target classpath contains a usable deserialization gadget chain.

At review time, Apache issue #4255 existed, was closed, and had a deleted/unavailable body via the GitHub API. The issue comments referenced archive and PoC repository links, and no CVE or Apache security advisory was observed. Lost Boys Cyber therefore treats this as an emerging, high-priority exposure candidate rather than an Apache-confirmed CVE. It is materially different from Log4Shell: ordinary applications that only write local logs, JSON logs, or syslog output are not directly exposed unless they also run a native Java serialized log-event receiver such as ObjectInputStreamLogEventBridge-like code or legacy serialized TcpSocketServer paths.

5. Attack Scenarios

Scenario	Preconditions	Attacker Action	Likely Outcome	Defensive Priority
JSON parser disruption	Application uses JsonTemplateLayout or MapMessage JSON serialization and logs attacker-controlled floating-point values	Submit NaN/Infinity/-Infinity through a field that is logged	Malformed JSON; SIEM/parser rejection or corrupted record	Patch log4j-api/layout-template-json and monitor parser failures
XML event suppression	Application uses XmlLayout or Log4j1XmlLayout and logs attacker-controlled messages/MDC values	Inject XML 1.0 forbidden characters	Malformed XML or appender delivery failure depending on StAX implementation	Patch log4j-core/log4j-1.2-api; review status logger errors
Syslog CRLF injection	Application directly configures Rfc5424Layout with affected attributes	Place CRLF sequences in user-controlled logged fields	Forged/confusing syslog records or split event boundaries	Upgrade and validate newline/framing configuration
Remote log interception	Socket/Syslog/SMTP appender uses TLS through nested Ssl config and broad trust roots	MITM remote logging channel with a trusted CA-issued certificate	Confidential log exposure or traffic redirection	Upgrade; restrict trust stores; validate certificate hostname checks in staging
Detection evasion around intrusion	Any affected structured layout logs security-relevant request data	Combine exploit attempt or credential abuse with malformed log payload	Security-relevant record may fail parsing or disappear from normal dashboards	Hunt raw logs, parser dead-letter queues, and ingestion error telemetry
Serialized LogEvent receiver RCE	Application exposes native Java serialized LogEvent receiver and has a usable gadget chain on classpath	Send crafted serialized payload using MarshalledObject bypass pattern	Possible code execution or resource-exhaustion DoS in receiver process	Disable/segment receiver, block Java serialization, apply JVM serial filter, and hunt for unexpected listener exposure

6. Threat Landscape & Exploitation Status

Public reporting reviewed for this report supports patch urgency and exposure triage but does not support claims of mass exploitation comparable to Log4Shell. These issues are conditional: exploitation depends on application logging patterns, configured layout/appender choices, downstream parser strictness, network position, and trust-store configuration.

That conditional nature should not lead defenders to ignore the cluster. Logging libraries sit in the evidence path for incident response. Weaknesses that break log structure, enable record injection, or expose remote logging channels can reduce confidence in audit trails, alert fidelity, and forensic reconstruction.

7. Affected Products and Exposure

Exposure Area	What to Inventory	Risk Signal	Recommended Disposition
Java applications and containers	Maven/Gradle dependencies for log4j-api, log4j-core, log4j-layout-template-json, log4j-1.2-api	Versions below fixed targets or multiple Log4j versions in shaded JARs	Upgrade and rebuild images; verify SBOM output
Structured JSON logging	JsonTemplateLayout, message resolver, MapMessage/ObjectMessage logging	Parser failures after unusual numeric values; APIs accepting floats	Upgrade to 2.25.5 or 2.26.1; sanitize/normalize non-finite values at app boundary
Structured XML logging	XmlLayout, Log4j1XmlLayout, MDC-heavy apps	XML fatal parse errors, Log4j status logger exceptions	Upgrade to 2.25.4+; prefer non-deprecated layouts where possible
Syslog forwarding	Direct Rfc5424Layout configuration, TCP/TLS framing, newline settings	CRLF in log content; framing mismatch; unexpected RFC6587 records	Upgrade to 2.25.4+; test newline escaping and TLS framing
Remote appender TLS	SocketAppender, SyslogAppender, SMTP appender with nested Ssl	Broad Java trust store; no private CA; verifyHostName assumed but not effective	Upgrade; restrict trust root; validate cert hostname checks in staging
Legacy bridge usage	log4j-1.2-api compatibility layer	org.apache.log4j.xml.XMLLayout or Log4j 1 migration bridge	Remove bridge dependency; migrate config to native Log4j 2
Serialized log receivers	ObjectInputStreamLogEventBridge, TcpSocketServer serialized receiver paths, custom services reading serialized LogEvent objects	Listening ports receiving Java serialization stream magic 00 05; no authentication/TLS; gadget libraries on classpath	Disable or isolate; migrate to JSON/syslog/OTLP; add JVM serialization filters and network ACLs

8. Mitigation and Workarounds

Priority	Action	Notes
P0	Upgrade log4j-api to 2.25.5 or 2.26.1 where applicable	Required for CVE-2026-49844; prefer this over stopping at 2.25.4 if JSON MapMessage paths exist
P0	Upgrade log4j-core, log4j-layout-template-json, and log4j-1.2-api to 2.25.4+ at minimum	Covers April 2026 cluster CVE-2026-34477 through CVE-2026-34481

P1	Eliminate deprecated Log4j 1-to-2 bridge usage	Reduces exposure to bridge-specific layout issues and prepares for Log4j 3 removal
P1	Review logging configurations for JsonTemplateLayout, XmlLayout, Rfc5424Layout, SocketAppender, SyslogAppender, and SMTP appenders	These are the exposure pivots that turn dependency presence into vulnerability relevance
P1	Restrict remote appender trust stores to a private CA or pinned receiver trust root	Reduces impact if immediate library upgrade is delayed
P2	Route parser failures and dead-letter queues into alerts	Detects both exploitation attempts and accidental data-loss conditions
P2	Add regression tests for logged user-controlled edge cases	Include NaN/Infinity, CRLF, XML forbidden characters, and MDC values
P0 for exposed receivers	Disable native Java serialized LogEvent receivers exposed to untrusted networks	Especially relevant to issue #4255-style exposure; prefer JSON/syslog/OTLP over Java native serialization
P1	On JDK 9+, evaluate -Djdk.serialFilter rules that reject java.rmi.MarshalledObject where compatible	Test first: this can break legitimate serialized log forwarding. JDK 8 lacks the modern global filter behavior and needs architectural mitigation.

Temporary mitigations are configuration-specific and less reliable than upgrading. For CVE-2026-34477 on 2.25.3, setting `-Dlog4j2.sslVerifyHostName=true` may engage the system-property pathway, but this should be treated as a stopgap and not a substitute for 2.25.4+.

9. Detection Engineering

9.1. Detection Summary

Signal	Telemetry Source	Analytic Goal
Vulnerable Log4j Maven coordinates in build or runtime images	SBOM, dependency manifests, container inventory	Find vulnerable artifacts even when packaged in shaded JARs
Affected layout/appender strings in configuration	Source repos, deployed config files, Kubernetes ConfigMaps, application packages	Separate vulnerable dependency presence from exploitable configuration
Log parser rejection spikes	SIEM/indexer/parser metrics, dead-letter queues	Detect malformed JSON/XML or CRLF abuse
Log4j StatusLogger exceptions around XML/JSON serialization	Application logs and stderr	Identify event-loss conditions caused by layout serialization failures
Remote logging TLS anomalies	Network logs, TLS inspection metadata, receiver logs	Detect certificate mismatch, unexpected endpoints, and remote log redirection
Java serialization listener exposure	NetFlow, EDR socket telemetry, service inventory, packet metadata	Identify services receiving native Java serialized streams and prioritize issue #4255 triage
title: Possible Log4j Structured Log Injection Or Parser Disruption Attempt id: 55a5af01-0c80-4d7b-9472-000000002026 status: experimental		


```

description: Detects strings commonly associated with Log4j2 2026 structured logging edge cases in web
or application logs. Tune field names to the local log schema.
references:
  - https://logging.apache.org/security.html#CVE-2026-49844
  - https://logging.apache.org/security.html#CVE-2026-34480
  - https://logging.apache.org/security.html#CVE-2026-34478
author: Lost Boys Cyber
date: 2026/08/26
logsource:
  category: webserver
detection:
  nonfinite_json:
    cs-uri-query|contains:
      - 'NaN'
      - 'Infinity'
      - '-Infinity'
    cs-user-agent|contains:
      - 'NaN'
      - 'Infinity'
      - '-Infinity'
  crlf_probe:
    cs-uri-query|contains:
      - '%0d%0a'
      - '%0D%0A'
      - '\r\n'
  xml_forbidden_probe:
    cs-uri-query|contains:
      - '%00'
      - '%01'
      - '%02'
      - '%08'
      - '%0B'
      - '%0C'
  condition: nonfinite_json or crlf_probe or xml_forbidden_probe
falsepositives:
  - Scientific or analytics APIs may legitimately receive NaN/Infinity-like strings; scope to
  applications that log request fields into Log4j structured layouts.
level: medium

// Microsoft Defender / Sentinel: vulnerable Log4j package and config exposure sweep from file
telemetry
DeviceFileEvents
| where Timestamp > ago(30d)
| where FileName matches regex @"log4j-(api|core|layout-template-json|1\.2-api)-2\.(1[3-9]|2[0-5])\.[0-9]+\.\jar"
  or FileName has_any ("log4j2.xml", "log4j2.json", "log4j2.yaml", "log4j2.properties")
| extend ExposureHint = case(
  FileName has "layout-template-json", "JsonTemplateLayout component present",
  FileName has "log4j-core", "Log4j Core present",
  FileName has "1.2-api", "Log4j 1-to-2 bridge present",
  FileName startswith "log4j2", "Configuration file present",
  "Review required")
| project Timestamp, DeviceName, FolderPath, FileName, ExposureHint, InitiatingProcessFileName,
InitiatingProcessCommandLine

// Microsoft Defender: hunt for Java processes exposing likely native-serialization receiver ports
DeviceNetworkEvents
| where Timestamp > ago(14d)
| where InitiatingProcessFileName in~ ("java.exe", "java", "javaw.exe")
| where ActionType has_any ("Listening", "ConnectionSuccess")
| where LocalPort in (4560, 4561, 4712, 4713) or RemotePort in (4560, 4561, 4712, 4713)
| project Timestamp, DeviceName, InitiatingProcessFileName, InitiatingProcessCommandLine, LocalIP,
LocalPort, RemoteIP, RemotePort, ActionType
index=* ("log4j" OR "JsonTemplateLayout" OR "XmlLayout" OR "Rfc5424Layout" OR "StatusLogger")
("NaN" OR "Infinity" OR "-Infinity" OR "%0d%0a" OR "%0D%0A" OR "not valid JSON" OR "XML" OR "forbidden
character" OR "parse" OR "parser")
| stats count min(_time) as first_seen max(_time) as last_seen values(source) as sources

```

```
values(sourcetype) as sourcetypes by host
| convert ctime(first_seen) ctime(last_seen)
```

10. Threat Hunting

10.1. Hunt Hypotheses

Hypothesis	Rationale	Primary Data
An attacker tried to hide a request or authentication event by forcing malformed structured log output	JSON/XML parser rejection can remove records from normal dashboards	Raw web/application logs, parser dead-letter queues, SIEM ingestion metrics
A vulnerable service has Log4j dependencies but only a subset is exploitable due to configuration	Dependency scanners over-alert unless layout/appender context is checked	SBOM, file inventory, repo configuration, runtime config maps
Remote logging traffic can be intercepted because the appender trusts broad public CAs	CVE-2026-34477/68161 require network position and trusted certificate presentation	Network paths, TLS receiver certificates, Java trust stores, appender configs
Deprecated Log4j bridge usage persists after migration projects	log4j-1.2-api remains common in legacy Java apps	Maven/Gradle manifests, JAR names, XML layout class names
Serialized LogEvent receivers are reachable from untrusted networks	Issue #4255-style RCE requires a receiver service and classpath gadgets	Listening socket inventory, firewall rules, packet captures, Java process command lines

```
# Linux/container filesystem triage: find Log4j2 JARs and affected config pivots.
# Run inside an application image, container mount, or host path. Review output manually; do not delete
files from production hosts.
root="${1:-/}"
find "$root" -type f \( -name 'log4j-*.jar' -o -name 'log4j2.*' -o -name '*.properties' -o -name
 '*.xml' \) 2>/dev/null |
while read -r path; do
  case "$path" in
    *log4j-api-2.*.jar|*log4j-core-2.*.jar|*log4j-layout-template-json-2.*.jar|*log4j-1.2-api-2.*.jar)
      printf 'JAR,%s\n' "$path" ;;
    *log4j2.*)
      if grep -Eiq 'JsonTemplateLayout|XmlLayout|Log4j1XmlLayout|Rfc5424Layout|SocketAppender|
SyslogAppender|SMTP|ObjectInputStreamLogEventBridge|TcpSocketServer|createSerializedSocketServer|
SerializedLayout|<Ssl|verifyHostName|newLineEscape|useTlsMessageFormat' "$path"; then
        printf 'CONFIG_PIVOT,%s\n' "$path"
        grep -Ein 'JsonTemplateLayout|XmlLayout|Log4j1XmlLayout|Rfc5424Layout|SocketAppender|
SyslogAppender|SMTP|ObjectInputStreamLogEventBridge|TcpSocketServer|createSerializedSocketServer|
SerializedLayout|<Ssl|verifyHostName|newLineEscape|useTlsMessageFormat' "$path" | head -20
      fi ;;
    esac
  done

// Hunt for suspicious web requests followed by ingestion or app errors on the same host.
let probes = dynamic(["NaN", "Infinity", "-Infinity", "%0d%0a", "%0D%0A", "%00", "%01", "%02", "%08",
"%0B", "%0C"]);
let ProbeEvents = union isfuzzy=true AppServiceHTTPLogs, W3CIISLog
| where TimeGenerated > ago(14d)
| extend RawText=tostring(pack_all())
| where RawText has_any (probes)
| project ProbeTime=TimeGenerated, Computer, RawText;
let ErrorEvents = union isfuzzy=true AppTraces, Syslog, Event
| where TimeGenerated > ago(14d)
| extend ErrorText=tostring(pack_all())
| where ErrorText has_any ("JsonTemplateLayout", "MapMessage", "XmlLayout", "Rfc5424Layout",
"StatusLogger", "not valid JSON", "forbidden character", "parse error")
| project ErrorTime=TimeGenerated, Computer, ErrorText;
```

```

ProbeEvents
| join kind=innerunique ErrorEvents on Computer
| where ErrorTime between (ProbeTime .. ProbeTime + 30m)
| project ProbeTime, ErrorTime, Computer, RawText, ErrorText

```

11. MITRE ATT&CK Mapping

Technique	Name	Why It Applies
T1190	Exploit Public-Facing Application	User-controlled input into web/API services is a likely delivery path for malformed values and CRLF/XML probes
T1562.008	Impair Defenses: Disable or Modify Cloud Logs	The operational effect is telemetry impairment, even when achieved through application logging edge cases rather than cloud-control-plane changes
T1070.002	Indicator Removal on Host: Clear Linux or Mac System Logs	Related forensic objective: reduce or confuse audit trails; use as conceptual mapping, not direct evidence of file deletion
T1040	Network Sniffing	TLS appender hostname verification issues require network-positioned interception to expose remote logging traffic
T1557	Adversary-in-the-Middle	CVE-2026-34477/68161 are MITM-style trust validation failures for remote log transmission
T1059	Command and Scripting Interpreter	If issue #4255 is exploitable with a gadget chain, post-deserialization payloads may execute OS commands in the Java process context
T1499	Endpoint Denial of Service	Object-graph bomb payloads may exhaust CPU or memory in native Java deserialization receivers

12. Validation / Lab Testing

Recommended validation is safe and non-exploitative: build a staging Java service using the affected layout/appender configuration and verify whether fixed Log4j versions preserve valid structured output.

Test	Expected Vulnerable Behavior	Expected Fixed Behavior
Log MapMessage containing Double.NaN with JsonTemplateLayout message resolver	JSON contains bare NaN or parser rejects record	JSON remains RFC 8259-compliant
Log user-controlled value with XML 1.0 forbidden character through XmlLayout	Malformed XML or StatusLogger exception; record missing from appender	Character sanitized; record delivered
Send user field containing CRLF to Rfc5424Layout direct configuration	Split or injected syslog-like lines under affected config	Newline escaping/framing preserved

Connect Socket/Syslog/SMTP appender over TLS to certificate with mismatched hostname but trusted CA	Connection may succeed despite verifyHostName attribute expectation	Hostname mismatch rejected
In a closed lab, send benign Java serialization stream markers to any suspected serialized LogEvent receiver	Receiver accepts native serialization protocol on an exposed port	Receiver disabled, authenticated/segmented, or migrated away from Java serialization

Do not run these tests against production logging pipelines. The test objective is to validate dependency and configuration remediation, not to create alert gaps in live SIEM ingestion.

13. Recommended Actions Summary

Timeframe	Owner	Action
Today	AppSec / Platform	Inventory Log4j2 artifacts and affected layout/appender configuration pivots across source, CI, containers, and runtime hosts
Today	Engineering	Upgrade log4j-api to 2.25.5 or 2.26.1 where JSON MapMessage paths exist; upgrade core/layout/bridge components to 2.25.4+ minimum
Today	Platform / Infrastructure	Search for exposed native Java serialized LogEvent receivers and disable or segment them while issue #4255 remains unassigned/unpatched
24-72 hours	Detection Engineering	Deploy dependency/config sweeps and parser-failure hunts; monitor dead-letter queues and StatusLogger errors
24-72 hours	Infrastructure	Validate remote logging TLS trust stores and certificate hostname enforcement for Socket/Syslog/SMTP appenders
7 days	Engineering Leadership	Remove deprecated log4j-1.2-api bridge dependencies and add regression tests for structured logging edge cases
Ongoing	SOC / IR	Treat missing or malformed logs around suspicious requests as a detection signal, not just an ingestion nuisance

14. Indicators, Observables, and Source Limitations

No unique malicious infrastructure, file hashes, domains, or IP addresses were published in the reviewed authoritative sources. The useful observables are configuration and behavior indicators:

Observable	Type	Operational Use
org.apache.logging.log4j:log4j-api versions 2.13.1-2.25.4 or 2.26.0	Vulnerable component	Patch prioritization for CVE-2026-49844
log4j-layout-template-json versions 2.14.0-2.25.3	Vulnerable component	Patch prioritization for CVE-2026-34481

log4j-core versions below 2.25.4 with XmlLayout/Rfc5424Layout/network appenders	Vulnerable component + config	Exposure confirmation
log4j-1.2-api bridge versions below 2.25.4 with Log4j1XmlLayout	Vulnerable component + config	Legacy migration priority
NaN, Infinity, -Infinity in attacker-controlled logged fields	Behavioral indicator	Potential JSON disruption attempt
Encoded CRLF such as %0d%0a or literal \r\n in logged request fields	Behavioral indicator	Potential syslog/log injection attempt
XML 1.0 forbidden control characters in logged fields	Behavioral indicator	Potential XML parser disruption attempt
Parser error spikes, dead-letter queue growth, or StatusLogger exceptions	Telemetry health indicator	Evidence of accidental or malicious event-loss condition
ObjectInputStreamLogEventBridge, TcpSocketServer, createSerializedSocketServer, SerializedLayout	Configuration/code indicator	Potential issue #4255-style exposure requiring receiver triage
Java serialization stream magic ac ed 00 05 to Java listener	Network indicator	Possible native serialization probing; inspect only in authorized telemetry

15. References

- [1] Apache Logging Services Security: <https://logging.apache.org/security.html>
- [2] Apache Log4j 2 Security: <https://logging.apache.org/log4j/2.x/security.html>
- [3] Apache CycloneDX Vulnerability Disclosure Report: <https://logging.apache.org/cyclonedx/vdr.xml>
- [4] CVE-2026-49844 record: <https://cveawg.mitre.org/api/cve/CVE-2026-49844>
- [5] CVE-2026-34477 record: <https://cveawg.mitre.org/api/cve/CVE-2026-34477>
- [6] CVE-2026-34478 record: <https://cveawg.mitre.org/api/cve/CVE-2026-34478>
- [7] CVE-2026-34479 record: <https://cveawg.mitre.org/api/cve/CVE-2026-34479>
- [8] CVE-2026-34480 record: <https://cveawg.mitre.org/api/cve/CVE-2026-34480>
- [9] CVE-2026-34481 record: <https://cveawg.mitre.org/api/cve/CVE-2026-34481>
- [10] CVE-2025-68161 record: <https://cveawg.mitre.org/api/cve/CVE-2025-68161>
- [11] Apache Log4j JsonTemplateLayout manual: <https://logging.apache.org/log4j/2.x/manual/json-template-layout.html>
- [12] Apache Log4j Layouts manual: <https://logging.apache.org/log4j/2.x/manual/layouts.html>
- [13] Apache Log4j Network Appenders manual: <https://logging.apache.org/log4j/2.x/manual/appenders/network.html>
- [14] Apache Log4j 1 migration guide: <https://logging.apache.org/log4j/2.x/migrate-from-log4j1.html>
- [15] Apache logging-log4j2 issue #4255: <https://github.com/apache/logging-log4j2/issues/4255>
- [16] The CyberSec Guru same-day article: <https://thecybersecguru.com/news/log4j2-deserialization-vulnerability-rce/>
- [17] Example public PoC reference linked from issue comments: <https://github.com/dinosn/log4j-4255>